

claude-windows / e83b09d8-e113-4d4d-9411-cd80ae217af9

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\e83b09d8-e113-4d4d-9411-cd80ae217af9\subagents\workflows\wf\_ae339437-da9\agent-a720b660798f2cfad.jsonl`
- SHA-256: `9ce2981d4997cee0db77273ef24fb27559561c0f6eeb4a9649e438e7730c5d68`
- Source modified: `2026-07-07T15:52:05+00:00`
- Imported at: `2026-07-08T15:59:35+00:00`
- project: `wf\_ae339437-da9`
- session_id: `e83b09d8-e113-4d4d-9411-cd80ae217af9`

Transcript

1. user (2026-07-07T15:48:13.158Z)

You are reviewing ONE commit on branch `fix/cloud-run-exec-status-fastfail` in the repo at:

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
Always start by: `cd "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer"` (the Bash cwd may default elsewhere). Use Read/Grep with absolute paths under that repo.

The commit hardens `CloudRunCompute.run\_infer` (backend/compute/cloud_run.py): the done-marker stays the
SOLE success signal, but each bucket-poll ALSO probes the Cloud Run Executions API so a worker that
TERMINATED without ever writing a marker (crash/OOM/an old image argparse-aborting on an unknown flag)
fails fast (raise RemoteExecutionFailed) instead of polling the full remote_poll_max_wait_sec (~65 min).
`\_dispatch\_remote` maps RemoteExecutionFailed -> rc 5 (distinct from the rc 4 poll-timeout).

CRITICAL CONTEXT: this commit was REBASED onto a master that had since merged two sibling PRs touching
the same files:
- #513 (23c1868): added `ComputeJobSpec.skip\_validation` + a `--skip-validation` worker flag;
the

control-plane dispatcher sets it so the worker skips redundant re-validation.
- #512 (20862d7): a validators shared-decode refactor (file-disjoint from this commit).
The motivating scenario for THIS fix: a post-#513 control-plane dispatching `--skip-validation`
to an
OLD worker image (no such flag) -> the worker argparse-aborts (exit 2) BEFORE writing a marker
-> the
exact ~65-min hang this fix collapses to a prompt rc-5 failure.

To see EXACTLY what this commit changes (only 5 files), run:

```
git diff origin/master...HEAD      (three-dot: my change vs merge-base)
To see the sibling context that landed on the same files, run:
git diff 3be0935..origin/master -- backend/compute/base.py backend/compute/cloud_run.py
backend/worker/__main__.py
```

Read the FULL current files (not just the diff) to judge correctness in context:

```
backend/compute/cloud_run.py, backend/compute/base.py, backend/compute/__init__.py,
backend/worker/__main__.py, backend/orchestration.py (the API caller),
tests/test_compute_backend.py,
backend/infrastructure/execution_policy.py (poll_until semantics).
```

Report ONLY real, defect-grade findings with a concrete failure scenario. Do NOT report style nits,
things the gate already covers, or speculation. If the change is correct on your lens, return an

```

empty
findings array and say so in `assessment`. Full suite (1833) + ruff + mypy + coverage 85.66%
already pass;
do not re-report "add a test" unless a SPECIFIC untested behavior would plausibly break.

LENS ? FAST-FAIL CONTROL FLOW. Scrutinize _poll_check / _terminal_execution_state / run_infer /
_dispatch_remote
and the orchestration.py API caller. Verify: (1) the marker is checked FIRST and always wins
(incl. an empty-dict
marker) even if status later flips; (2) the post-status race re-check is correct; (3)
RemoteExecutionFailed
(a RuntimeError, NOT a PolicyTimeout) actually PROPAGATES out of poll_until and PAST the `except
PolicyTimeout`
in run_infer ? confirm poll_until does not swallow check() exceptions; (4) rc 5 vs rc 4
semantics are coherent and
the orchestration.py `except Exception` path still yields a fast, labelled failure; (5) the
best-effort swallow of
a raising status probe cannot fail a healthy run NOR mask a real terminal failure. Flag any
ordering/exception bug.

```

2. assistant (2026-07-07T15:48:15.824Z)

I'll start by examining the repo state and the commit under review.

3. user (2026-07-07T15:48:18.560Z)

```

145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
---BRANCH---
fix/cloud-run-exec-status-fastfail
---STATUS---
?? score_golden_gate.py
?? score_local.py

```

4. assistant (2026-07-07T15:48:20.903Z)

I'll begin by looking at the exact diff of this commit and the sibling context.

5. user (2026-07-07T15:48:23.279Z)

```

backend/compute/__init__.py | 8 +-
backend/compute/base.py     | 14 +++
backend/compute/cloud_run.py | 115 ++++++
backend/worker/__main__.py   | 16 ++-
tests/test_compute_backend.py | 229 ++++++
5 files changed, 375 insertions(+), 7 deletions(-)
=====FULL DIFF=====
diff --git a/backend/compute/__init__.py b/backend/compute/__init__.py
index 1091582..482f3c9 100644
--- a/backend/compute/__init__.py
+++ b/backend/compute/__init__.py
@@ -13,13 +13,19 @@ import dataclasses
 import os
 from typing import Any, Optional

-from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
+from backend.compute.base import (
+    ComputeBackend,
+    ComputeJobSpec,
+    ComputeResult,

```

```

+ RemoteExecutionFailed,
+)
from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy

__all__ = [
    "ComputeBackend",
    "ComputeJobSpec",
    "ComputeResult",
+   "RemoteExecutionFailed",
    "get_compute_backend",
]

diff --git a/backend/compute/base.py b/backend/compute/base.py
index 31af3fe..3ad76fc 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -17,6 +17,20 @@ from dataclasses import dataclass, field
    from typing import Callable, Optional

+class RemoteExecutionFailed(RuntimeError):
+    """A dispatched remote job's execution reached a TERMINAL state without ever writing its
+    completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image argparse-
+    rejecting
+    an unknown flag) before the marker push, or the marker upload itself failed on an otherwise
+    Succeeded run.
+
+    Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL be
+    running): this says the execution is DEFINITELY over and no marker will ever appear, so
+    the
+    dispatcher must stop bucket-polling immediately instead of burning the full
+    ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message carries the
+    job + execution name + terminal state + the ``gcloud`` command to read the execution log,
+    so
+    the whole "worker crashed before the done-marker" class fails fast AND diagnostically."""
+
+    @dataclass(frozen=True)
+    class ComputeJobSpec:
+        """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will (re)compute
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 7247ace..1e6e313 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -11,6 +11,10 @@ Flow (the cloud-serving compute path):
    job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-but-
    healthy
    worker that finished just past the deadline is rescued, not wasted ? then best-effort
    CANCELS
    the still-running execution so the L4 doesn't keep billing until ``--task-timeout``.
+    *Fast negative path:* while the marker stays the SOLE success signal, each poll also
+    probes the
+    Cloud Run Executions API ? if the execution has TERMINATED without ever writing a marker
+    (a
+    worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
    diagnostically
+    (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile wait.
    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

@@ -32,7 +36,12 @@ import uuid
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional

-from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult

```

```

+from backend.compute.base import (
+    ComputeBackend,
+    ComputeJobSpec,
+    ComputeResult,
+    RemoteExecutionFailed,
+)
+    from backend.infrastructure.execution_policy import (
+        DEFAULT_POLICY,
+        ExecutionPolicy,
+
@@ -76,6 +85,21 @@ class CloudRunJobClient(ABC):
+    May raise ? the caller treats every failure as best-effort (the original poll timeout
+
+    is
+        NEVER masked by a cancel error)."""
+
+    def execution_terminal_state(self, execution: str) -> Optional[str]:
+        """Return the execution's TERMINAL state as a short label (`"Failed"` /
+        `"Cancelled"` /
+        `"Succeeded"`) if it has finished, else `None` (still pending/running, or the state
+        can't be determined).
+
+        ADVISORY, not authoritative: it feeds `run_infer`'s fast negative path so a worker
+that
+        terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails fast
+instead
+        of bucket-polling the full budget. The done-marker stays the ONLY success signal.
+
+        NON-abstract with a default of `None` (additive / default-OFF): a client that can't ?
+or
+        chooses not to ? poll the Executions API degrades transparently to today's marker-only
+        polling. Implementations MAY raise on a transient API error ? `run_infer` treats a
+raise
+        as `None` (unknown ? keep waiting), so a flaky status API never fails a healthy
+run."""
+        return None
+
+
+    class GoogleCloudRunJobClient(CloudRunJobClient):
+        """Real client ? lazy-imports `google-cloud-run`. Owner-verified on the M7 real run; CI
+uses
@@ -145,6 +169,35 @@ class GoogleCloudRunJobClient(CloudRunJobClient):
+        ) # pragma: no cover - exercised only on a real run
+        client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
+
+    def execution_terminal_state(self, execution: str) -> Optional[str]:
+        # Advisory status probe (see the ABC docstring): describe the execution and, ONLY once
+it
+        # is terminal (`completion_time` set), classify it. A non-resolvable name / absent
+SDK
+        # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
+        # raises on bad input, because the probe is best-effort and must not fail a healthy
+run.
+        if "/executions/" not in (execution or ""):
+            return None
+
+        try:
+            from google.cloud import run_v2 # type: ignore[attr-defined]
+        except Exception: # pragma: no cover - real SDK not present in CI
+            return None
+
+        client = (
+            run_v2.ExecutionsClient()
+        ) # pragma: no cover - exercised only on a real run
+        exe = client.get_execution( # pragma: no cover - real run only
+            request=run_v2.GetExecutionRequest(name=execution)
+        )
+
+        # completion_time is a proto Timestamp: unset ? seconds==0 ? still running/pending.
+        if not getattr(getattr(exe, "completion_time", None), "seconds", 0):

```

```

+         return None # pragma: no cover - real run only
+         # Terminal: prefer the explicit per-task counts (a cancel shows as cancelled_count>0).
+         if getattr(exe, "cancelled_count", 0): # pragma: no cover - real run only
+             return "Cancelled"
+         if getattr(exe, "failed_count", 0): # pragma: no cover - real run only
+             return "Failed"
+         if getattr(exe, "succeeded_count", 0): # pragma: no cover - real run only
+             return "Succeeded"
+         return "Failed" # pragma: no cover - terminal but no positive success ? treat as
failed
+

class CloudRunCompute(ComputeBackend):
    """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
@@ -215,7 +268,7 @@ class CloudRunCompute(ComputeBackend):

    try:
        marker = poll_until(
-         lambda: self._read_marker(done_uri),
+         lambda: self._poll_check(done_uri, str(execution or "")),
            self._policy,
            label="cloud-run-infer",
            logger=LOG,
@@ -224,6 +277,9 @@ class CloudRunCompute(ComputeBackend):
            on_tick=on_wait,
        )
    except PolicyTimeout as timeout_exc:
+         # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
+         # so it propagates past here uncaught ? the dispatch handler maps it to a distinct,
+         # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.

        marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
        video_id = str(marker.get("video_id", ""))
        # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
@@ -249,6 +305,61 @@ class CloudRunCompute(ComputeBackend):
            execution=str(execution or ""),
        )

+     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
+         """One bucket-poll iteration, hardened with an Executions-API fast negative path.
+
+         The done-marker is the SOLE success signal and is checked FIRST ? a present marker
always
+         wins (even if the execution's status later flips), so a worker that wrote a SUCCESS *or
a
+         FAILURE* marker resolves through the normal path with its real returncode. Only when
the
+         marker is absent do we consult the execution status: a worker that TERMINATED without
ever
+         writing a marker (crash / OOM / an old image argparse-aborting on an unknown flag)
would
+         otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min) before
failing
+         opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
+         failure.
+
+         Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
+         ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over, no
marker
+         will ever appear)."""
+         marker = self._read_marker(done_uri)
+         if marker is not None:

```

```

+         return marker # success signal ? authoritative even if status later flips
+         state = self._terminal_execution_state(execution)
+         if state is None:
+             return None # still pending/running, or status unknown ? keep bucket-polling
+         # The execution is terminal but no marker is present. Re-check the marker ONCE to close
the
+         # race where the worker wrote it between our two reads (mirrors the post-timeout rescue
in
+         # _handle_poll_timeout). If it is genuinely absent, the process is gone and no marker
will
+         # EVER appear ? fail fast + diagnostically rather than wait out the whole poll budget.
+         marker = self._read_marker(done_uri)
+         if marker is not None:
+             return marker
+         raise RemoteExecutionFailed(
+             f"cloud-run: job={self._job_name} execution={execution or '<unknown>'} reached "
+             f"terminal state {state!r} with NO completion marker ? the worker exited before "
+             f"writing one (crash / OOM / pre-marker abort, or a failed marker upload on an "
+             f"otherwise-Succeeded run). The execution is over, so the marker will never appear;
"
+             "inspect the Cloud Run Job execution log to diagnose: "
+             f"`gcloud run jobs executions describe {execution or '<execution>'} "
+             f"--region {self._region}`."
+         )
+
+ def _terminal_execution_state(self, execution: str) -> Optional[str]:
+     """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A raising
or
+     flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
+     so any exception ? and an empty/unresolvable execution name ? degrades to ``None``
(unknown
+     ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
+     if not execution:
+         return None
+     try:
+         return self._client.execution_terminal_state(execution)
+     except Exception: # noqa: BLE001 ? advisory probe: never let it kill the bucket-poll
+         LOG.debug(
+             "cloud-run: execution status probe failed for %s ? treating as unknown",
+             execution,
+             exc_info=True,
+         )
+         return None
+
+     def _handle_poll_timeout(
+         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
+     ) -> dict:
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 9b19215..6d3420b 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -176,9 +176,14 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
    container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
dispatch.

- A remote job that never writes a completion marker (a failed/hung container) makes the
bucket
- poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4 (remote
dispatch
- timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
+ A remote job that never writes a completion marker exits via one of two CLEAN codes

```

```

(distinct
+   from the local 2/3), never a raw traceback:
+   * rc 4 ? the bucket poll exhausted its budget (`PolicyTimeout`): the execution may
still be
+   running (the shim best-effort cancels it).
+   * rc 5 ? the execution has TERMINATED without a marker (`RemoteExecutionFailed`): a
worker
+   crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead of the
+   ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
+   from backend.compute import RemoteExecutionFailed
+   from backend.infrastructure.execution_policy import PolicyTimeout

    spec = ComputeJobSpec(
@@ -190,6 +195,11 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    )
    try:
        result = compute.run_infer(spec)
+   except RemoteExecutionFailed as e:
+       logger.error(
+           "[worker] remote execution terminated without a completion marker: %s", e
+       )
+       return 5
    except PolicyTimeout as e:
        logger.warning(
            "[worker] remote compute did not complete (no marker within budget): %s", e
diff --git a/tests/test_compute_backend.py b/tests/test_compute_backend.py
index faa4981..07e8068 100644
--- a/tests/test_compute_backend.py
+++ b/tests/test_compute_backend.py
@@ -12,7 +12,11 @@ import tempfile

import pytest

-from backend.compute import ComputeJobSpec, get_compute_backend
+from backend.compute import (
+    ComputeJobSpec,
+    RemoteExecutionFailed,
+    get_compute_backend,
+)
from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
from backend.config import AppConfig
from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout
@@ -110,6 +114,37 @@ class _NeverDoneClient(CloudRunJobClient):
    raise self._cancel_error

+class _CrashedNoMarkerClient(CloudRunJobClient):
+    """Simulates the bug's failure class: the worker execution TERMINATES (crash / OOM / an old
+    image argparse-aborting on an unknown flag) WITHOUT ever writing a done-marker. Dispatch
    returns
+    a resolvable execution name, no marker is ever written, and the Executions API reports a
    terminal
+    state ? so run_infer must fail FAST via the status probe instead of spinning the poll
    budget."""
+
+    EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-crash"
+
+    def __init__(self, *, state: str = "Failed"):
+        self.calls: list[dict] = []
+        self.cancelled: list[str] = []
+        self.status_polls = 0
+        self._state = state
+
+    def run_job(self, job_name, argv, *, region, project=None):
+        self.calls.append(

```

```

+         {"job": job_name, "argv": list(argv), "region": region, "project": project}
+     )
+     return self.EXEC_NAME
+
+     def cancel_execution(self, execution):
+         self.cancelled.append(execution)
+
+     def execution_terminal_state(self, execution):
+         self.status_polls += 1
+         # Anti-hang guard: the fast path MUST abort on the first terminal observation. If a
+         # regression kept polling despite a terminal status, trip here instead of spinning.
+         assert self.status_polls <= 5, "status probed repeatedly ? fast path did not abort the
poll"
+         return self._state
+
+
+     # ----- factory (default-OFF)

@@ -423,6 +458,198 @@ def test_poll_timeout_late_marker_rescues_result(monkeypatch):
    assert exists_calls["n"] == 2 # exactly ONE extra post-timeout check

+## ----- execution-status fast negative path (worker crashed pre-
+## marker)
+
+## poll_every_n_sec=0.0 ? the poll spins as fast as possible; max_wait_sec=2.0 BOUNDS any
+## regression
+## (a fast path that silently fell back to marker-only polling fails in ~2s here, never a 65-min
+## hang)
+## while the working fast path aborts on poll 0 ? instantly.
+_FAILFAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=2.0)
+
+def test_run_infer_fails_fast_when_execution_terminal_without_marker(tmp_path):
+    """The core fix: a worker execution that TERMINATED (Failed) without ever writing a marker
+is
+caught by the Executions-API probe ? RemoteExecutionFailed FAST (poll 0), not a ~65-min
+opaque
+hang. The diagnostic names the execution, the terminal state, and the log-inspection
+command."""
+    client = _CrashedNoMarkerClient(state="Failed")
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_FAILFAST,
+        token="tok",
+    )
+    with pytest.raises(RemoteExecutionFailed) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    msg = str(exc.value)
+    assert _CrashedNoMarkerClient.EXEC_NAME in msg # operator can find the execution
+    assert "Failed" in msg # the terminal state is surfaced
+    assert "gcloud run jobs executions describe" in msg # actionable log pointer
+    assert client.status_polls == 1 # aborted on the FIRST terminal observation (fast, not
+    spun)
+    assert client.cancelled == [] # a finished execution is never cancelled (nothing to
+    cancel)
+
+def test_dispatch_remote_maps_terminal_without_marker_to_rc5(tmp_path):
+    """The dispatch handler maps RemoteExecutionFailed to a distinct, clearly-labelled rc 5 ?

```

NOT

```
+ the rc 4 poll-timeout path (a still-maybe-running execution) and NOT a raw traceback ? so
+ 'worker crashed before the marker' is operationally distinguishable and surfaces
immediately."""
+ import argparse
+
+ from backend.worker import __main__ as wmain
+
+ client = _CrashedNoMarkerClient(state="Failed")
+ backend = CloudRunCompute(
+     run_client=client,
+     job_name="rally-worker",
+     region="asia-south1",
+     policy=_FAILFAST,
+     token="tok",
+ )
+ args = argparse.Namespace(
+     video_uri="gs://b/v.mp4",
+     out_uri=str(tmp_path / "bucket"),
+     segmenter="wasb_hybrid",
+     set=None,
+ )
+ rc = wmain._dispatch_remote(backend, args)
+ assert rc == 5 # the crashed-pre-marker fast-fail code...
+ assert rc != 4 # ...distinct from the poll-budget timeout
+ assert client.status_polls == 1 # reached the fast path (not the timeout branch)
+
+
+def test_succeeded_state_without_marker_also_fails_fast(tmp_path):
+    """Generalization: ANY terminal state with no marker is a guaranteed-futile wait. A
+    'Succeeded'
+    execution that nonetheless wrote no marker (the marker upload itself failed) also fails
+    fast,
+    with the state surfaced ? the whole 'execution over, no marker' class collapses, not just
+    Failed."""
+    client = _CrashedNoMarkerClient(state="Succeeded")
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_FAILFAST,
+        token="tok",
+    )
+    with pytest.raises(RemoteExecutionFailed) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert "Succeeded" in str(exc.value)
+
+
+def test_present_marker_wins_over_terminal_status(tmp_path):
+    """The done-marker stays AUTHORITATIVE: a worker that wrote a marker ? even a FAILURE
+    marker
+    (rc 2) ? resolves through the normal path with its REAL returncode, never a synthesized
+    RemoteExecutionFailed, even when the execution status also reads terminal. Status is a
+    negative
+    path only; it must never override a real marker (which is checked first)."""
+
+    class _MarkerAndFailedStatus(_FakeRunClient):
+        def execution_terminal_state(self, execution): # pragma: no cover - never reached
+            return "Failed" # the marker resolves poll 0, so this is never consulted
+
+    client = _MarkerAndFailedStatus(video_id="abc", returncode=2)
+    backend = CloudRunCompute(
+        run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
```

```

+ )
+ result = backend.run_infer(
+     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+ )
+ assert result.returncode == 2 # the worker's REAL rc, not RemoteExecutionFailed
+ assert result.video_id == "abc"
+
+
+def test_status_probe_error_is_swallowed_and_falls_back_to_marker_polling(tmp_path):
+    """Best-effort probe: a raising Executions API must NEVER fail a healthy run. The wait
+    degrades
+    to today's marker-only polling ? PolicyTimeout (the existing timeout path), NOT
+    RemoteExecutionFailed ? a flaky status API cannot manufacture a failure."""
+
+    class _RaisingStatusClient(_NeverDoneClient):
+        def execution_terminal_state(self, execution):
+            raise RuntimeError("executions API unreachable")
+
+    client = _RaisingStatusClient()
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_TIMEOUT_NOW,
+        token="tok",
+    )
+    with pytest.raises(PolicyTimeout): # swallowed probe error ? the unchanged timeout path
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert client.cancelled == [_NeverDoneClient.EXEC_NAME] # still cancels on the real
+    timeout
+
+
+def test_default_execution_terminal_state_is_none():
+    """The ABC default is None (additive / default-OFF): a client that doesn't override status
+    polling degrades transparently to marker-only behaviour, so every pre-existing fake is
+    unaffected and the seam never changes a healthy run's outcome."""
+    assert _NoopRunClient().execution_terminal_state("whatever") is None
+    assert (
+        _NeverDoneClient().execution_terminal_state(_NeverDoneClient.EXEC_NAME) is None
+    )
+
+
+def test_real_client_terminal_state_none_for_unresolvable_name():
+    """The real client's status probe is best-effort: a non-resolvable execution name returns
+    None
+    (can't probe ? keep marker-polling), NOT a raise ? exercisable in CI (the SDK is
+    absent)."""
+    from backend.compute.cloud_run import GoogleCloudRunJobClient
+
+    client = GoogleCloudRunJobClient()
+    for bogus in ("", "execution", "exec-123"):
+        assert client.execution_terminal_state(bogus) is None
+
+
+def test_terminal_status_with_marker_written_in_race_resolves_via_marker(monkeypatch):
+    """Concurrency race: the execution reads terminal exactly as the worker finishes writing
+    its
+    marker. _poll_check re-checks the marker ONCE after a terminal status ? if it landed in
+    between,
+    the completed result is ingested (the marker wins), NOT a spurious
+    RemoteExecutionFailed."""
+    from backend.compute import cloud_run as cr
+
+

```

```

+     exists_calls = {"n": 0}
+
+     class _FakeStore:
+         def exists(self, ref):
+             exists_calls["n"] += 1
+             # absent on the pre-status read, present on the ONE post-status race re-check
+             return exists_calls["n"] >= 2
+
+     monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
+
+     def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
+         if dst is None:
+             dst = os.path.join(tempfile.mkdtemp(prefix="cr-race-"), "m.json")
+             with open(dst, "w", encoding="utf-8") as f:
+                 f.write('{"video_id": "vrace", "returncode": 0, "segment_count": 1}')
+             return dst
+
+     monkeypatch.setattr(cr, "fetch", fake_fetch)
+     client = _CrashedNoMarkerClient(state="Failed") # status says terminal-Failed...
+     backend = cr.CloudRunCompute(
+         run_client=client, job_name="j", region="r", token="t", policy=_FAILFAST
+     )
+     result = backend.run_infer(
+         ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
+     )
+     assert result.video_id == "vrace" # ...but the marker that raced in WINS
+     assert exists_calls["n"] == 2 # exactly the pre-status read + the ONE race re-check
+     assert client.status_polls == 1 # status was consulted once, then the marker superseded it
+
+
+def test_terminal_execution_state_none_for_empty_execution():
+    """An empty/unknown execution name (run_job could not resolve one) can't be probed ? None
+    (degrade to marker-only polling), and the client's status API is never even called."""
+
+    class _Boom(_NoopRunClient):
+        def execution_terminal_state(self, execution): # pragma: no cover - must never be
called
+            raise AssertionError("must not probe an empty execution name")
+
+    backend = CloudRunCompute(run_client=_Boom(), job_name="j", region="r")
+    assert backend._terminal_execution_state("") is None
+
+
+    # ----- worker wiring: dispatch branch + done-
marker

```

6. assistant (2026-07-07T15:48:25.836Z)

Now let me read the full current files to judge correctness in context.

7. user (2026-07-07T15:48:27.340Z)

```

1     """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** `done_uri` via `execution_policy.poll_until` + the storage
layer (D16 ? the
8     completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so

```

```

9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10         cold-start is absorbed by the async poll. On a poll TIMEOUT (`max_wait_sec` can
undercut the
11         job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12         worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13         the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14         *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15         Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16         worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17         (`RemoteExecutionFailed`) instead of spinning the full ~65-min budget on a futile
wait.
18         3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19         ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21         The dispatched container is forced to run **INLINE** (`compute_target=local_gpu` +
native detector)
22         so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23
24         The Cloud Run trigger is an **injected client** (`CloudRunJobClient`) so the whole
shim is unit-tested
25         with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26         owner-verified on the M7 real run.
27         ""
28
29         from __future__ import annotations
30
31         import json
32         import logging
33         import os
34         import tempfile
35         import uuid
36         from abc import ABC, abstractmethod
37         from typing import Any, Callable, List, Optional
38
39         from backend.compute.base import (
40             ComputeBackend,
41             ComputeJobSpec,
42             ComputeResult,
43             RemoteExecutionFailed,
44         )
45         from backend.infrastructure.execution_policy import (
46             DEFAULT_POLICY,
47             ExecutionPolicy,
48             PolicyTimeout,
49             poll_until,
50         )
51         from backend.storage import fetch, get_storage, parse_uri
52
53         LOG = logging.getLogger("compute.cloud_run")
54
55         # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
56         # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
57         _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
58             "deployment.compute_target=local_gpu",

```

```

59         "indexing.detector_impl=native",
60     )
61
62
63     class CloudRunJobClient(ABC):
64         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
65
66         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
67         overrides the job's container ``args`` for this execution and starts it."""
68
69         @abstractmethod
70         def run_job(
71             self,
72             job_name: str,
73             argv: List[str],
74             *,
75             region: str,
76             project: Optional[str] = None,
77         ) -> str:
78             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
79             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
80
81         @abstractmethod
82         def cancel_execution(self, execution: str) -> None:
83             """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
84             times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
85             May raise ? the caller treats every failure as best-effort (the original poll
timeout is
86             NEVER masked by a cancel error)."""
87
88         def execution_terminal_state(self, execution: str) -> Optional[str]:
89             """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
90             ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
91             can't be determined).
92
93             ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
94             terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95             of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97             NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
98             chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99             polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
100             as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
101             return None
102
103
104     class GoogleCloudRunJobClient(CloudRunJobClient):
105         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
106         a fake, so this code path is never exercised in tests (the SDK isn't a test

```

```

dependency)."""
107
108     def run_job(
109         self,
110         job_name: str,
111         argv: List[str],
112         *,
113         region: str,
114         project: Optional[str] = None,
115     ) -> str:
116         # A None/empty project would build a bogus "projects/None/locations/..." path
117         # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
118         # in CI (where google-cloud-run is absent) and so mypy narrows project to str
119         if not project:
120             raise RuntimeError(
121                 "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
122                 "to resolve the job path; set it on the control plane (see
123                 )
124             try:
125                 from google.cloud import run_v2 # type: ignore[attr-defined]
126             except Exception as exc: # pragma: no cover - real SDK not present in CI
127                 raise RuntimeError(
128                     "google-cloud-run is required for CloudRunCompute's real client; install
129                     "control plane (it is intentionally NOT a CI/test dependency).")
130             ) from exc
131         client = (
132             run_v2.JobsClient()
133         ) # pragma: no cover - exercised only on the real M7 run
134         name = client.job_path(project, region, job_name)
135         overrides = run_v2.RunJobRequest.Overrides(
136             container_overrides=[
137                 run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
138             ]
139         )
140         op = client.run_job(
141             request=run_v2.RunJobRequest(name=name, overrides=overrides)
142         )
143         return (
144             getattr(op, "metadata", None)
145             and getattr(op.metadata, "name", "")
146             or "execution"
147         )
148
149     def cancel_execution(self, execution: str) -> None:
150         # run_job falls back to the literal string "execution" when the operation
151         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
152         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
153         # google-cloud-run is absent).
154         if "/executions/" not in (execution or ""):
155             raise RuntimeError(
156                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
157                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
158                 "one from the operation metadata, so there is nothing to cancel by
159                 name)."

```

```

159         )
160     try:
161         from google.cloud import run_v2 # type: ignore[attr-defined]
162     except Exception as exc: # pragma: no cover - real SDK not present in CI
163         raise RuntimeError(
164             "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
165             "control plane (it is intentionally NOT a CI/test dependency).")
166     ) from exc
167     client = (
168         run_v2.ExecutionsClient()
169     ) # pragma: no cover - exercised only on a real run
170     client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
171
172     def execution_terminal_state(self, execution: str) -> Optional[str]:
173         # Advisory status probe (see the ABC docstring): describe the execution and,
ONLY once it
174         # is terminal (`completion_time` set), classify it. A non-resolvable name /
absent SDK
175         # returns None (can't probe ? keep marker-polling) ? unlike run_job/cancel this
never
176         # raises on bad input, because the probe is best-effort and must not fail a
healthy run.
177         if "/executions/" not in (execution or ""):
178             return None
179         try:
180             from google.cloud import run_v2 # type: ignore[attr-defined]
181         except Exception: # pragma: no cover - real SDK not present in CI
182             return None
183         client = (
184             run_v2.ExecutionsClient()
185         ) # pragma: no cover - exercised only on a real run
186         exe = client.get_execution( # pragma: no cover - real run only
187             request=run_v2.GetExecutionRequest(name=execution)
188         )
189         # completion_time is a proto Timestamp: unset ? seconds==0 ? still
running/pending.
190         if not getattr(getattr(exe, "completion_time", None), "seconds", 0):
191             return None # pragma: no cover - real run only
192         # Terminal: prefer the explicit per-task counts (a cancel shows as
cancelled_count>0).
193         if getattr(exe, "cancelled_count", 0): # pragma: no cover - real run only
194             return "Cancelled"
195         if getattr(exe, "failed_count", 0): # pragma: no cover - real run only
196             return "Failed"
197         if getattr(exe, "succeeded_count", 0): # pragma: no cover - real run only
198             return "Succeeded"
199         return "Failed" # pragma: no cover - terminal but no positive success ? treat
as failed
200
201
202     class CloudRunCompute(ComputeBackend):
203         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
204
205     def __init__(
206         self,
207         *,
208         run_client: CloudRunJobClient,
209         job_name: str,
210         region: str,
211         project: Optional[str] = None,
212         storage_config: Optional[dict] = None,
213         policy: ExecutionPolicy = DEFAULT_POLICY,
214         token: Optional[str] = None,
215         container_overrides: Optional[tuple[str, ...]] = None,

```

```

216     ) -> None:
217         self._client = run_client
218         self._job_name = job_name
219         self._region = region
220         self._project = project
221         self._storage_config = storage_config or {}
222         self._policy = policy
223         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
224         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
225         self._token = token
226         self._container_overrides = (
227             _DEFAULT_CONTAINER_OVERRIDES
228             if container_overrides is None
229             else tuple(container_overrides)
230         )
231
232     def run_infer(
233         self,
234         spec: ComputeJobSpec,
235         *,
236         on_wait: "Callable[[float], None] | None" = None,
237     ) -> ComputeResult:
238         out_root = spec.out_uri.rstrip("/")
239         token = self._token or uuid.uuid4().hex
240         done_uri = f"{out_root}/_dispatch/{token}.done"
241         argv: List[str] = [
242             "infer",
243             "--video-uri",
244             spec.video_uri,
245             "--out-uri",
246             spec.out_uri,
247             "--done-uri",
248             done_uri,
249         ]
250         if spec.segmenter:
251             argv += ["--segmenter", spec.segmenter]
252         if spec.skip_validation:
253             # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
254             # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
255             argv.append("--skip-validation")
256         for kv in (*spec.config_overrides, *self._container_overrides):
257             argv += ["--set", kv]
258
259         execution = self._client.run_job(
260             self._job_name, argv, region=self._region, project=self._project
261         )
262         LOG.info(
263             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
264             self._job_name,
265             execution,
266             done_uri,
267         )
268
269         try:
270             marker = poll_until(
271                 lambda: self._poll_check(done_uri, str(execution or "")),
272                 self._policy,
273                 label="cloud-run-infer",
274                 logger=LOG,
275                 # Live heartbeat (#482): each still-waiting tick reaches the caller so
276                 # job.progress moves during the whole GPU run instead of freezing.

```

```

277         on_tick=on_wait,
278     )
279 except PolicyTimeout as timeout_exc:
280     # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
281     # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
282     # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
283     marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
284     video_id = str(marker.get("video_id", ""))
285     # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
286     # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
287     rc = int(marker.get("returncode", 1))
288     if not video_id:
289         LOG.warning(
290             "cloud-run: job=%s completion marker present but empty/unreadable (no "
291             "video_id) ? treating as failure",
292             self._job_name,
293         )
294     rc = rc or 1
295     outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
296     report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
297     LOG.info(
298         "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
299     )
300     return ComputeResult(
301         returncode=rc,
302         outputs_uri=outputs_uri,
303         video_id=video_id,
304         report=report,
305         execution=str(execution or ""),
306     )
307
308 def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
309     """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
310
311     The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
312     wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
313     FAILURE* marker resolves through the normal path with its real returncode. Only
when the
314     marker is absent do we consult the execution status: a worker that TERMINATED
without ever
315     writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
316     otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
317     opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
318     failure.
319
320     Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
321     ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
322     will ever appear)."""
323     marker = self._read_marker(done_uri)
324     if marker is not None:
325         return marker # success signal ? authoritative even if status later flips
326     state = self._terminal_execution_state(execution)
327     if state is None:

```

```

328         return None # still pending/running, or status unknown ? keep bucket-
polling
329         # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
330         # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
331         # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
332         # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
333         marker = self._read_marker(done_uri)
334         if marker is not None:
335             return marker
336         raise RemoteExecutionFailed(
337             f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
338             f"terminal state {state!r} with NO completion marker ? the worker exited
before "
339             "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
340             "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
341             "inspect the Cloud Run Job execution log to diagnose: "
342             f"`gcloud run jobs executions describe {execution or '<execution>'} "
343             f"--region {self._region}`."
344         )
345
346     def _terminal_execution_state(self, execution: str) -> Optional[str]:
347         """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
348         flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
349         so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
350         ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
351         if not execution:
352             return None
353         try:
354             return self._client.execution_terminal_state(execution)
355         except Exception: # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
356             LOG.debug(
357                 "cloud-run: execution status probe failed for %s ? treating as unknown",
358                 execution,
359                 exc_info=True,
360             )
361             return None
362
363     def _handle_poll_timeout(
364         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
365     ) -> dict:
366         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
367
368         The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
369         ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
370         ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
371         is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
372         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
373         execution name so the operator can verify the state in the GCP console. A cancel
failure

```

```

374         NEVER masks the original timeout."""
375         try:
376             late = self._read_marker(done_uri)
377         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
? failure
378             LOG.warning(
379                 "cloud-run: final post-timeout marker check failed ? treating as
absent",
380                 exc_info=True,
381             )
382             late = None
383         if late is not None:
384             LOG.warning(
385                 "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
386                 "? rescuing the completed result",
387                 self._job_name,
388                 execution,
389             )
390             return late
391         try:
392             self._client.cancel_execution(execution)
393         except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
394             cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
395             LOG.warning(
396                 "cloud-run: best-effort cancel of execution=%s failed: %s",
397                 execution,
398                 cancel_exc,
399                 exc_info=True,
400             )
401         else:
402             cancel_note = "cancel requested"
403             LOG.info("cloud-run: requested cancel of execution=%s", execution)
404         raise PolicyTimeout(
405             f"{timeout_exc} [cloud-run job={self._job_name} "
406             f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
407             "state in the GCP console]"
408         ) from timeout_exc
409
410     # --- storage-backed completion signal (bucket-poll) ---
411     def _read_marker(self, done_uri: str) -> Optional[dict]:
412         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
413         ref = parse_uri(done_uri)
414         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
415             return None
416         return (
417             self._read_json(done_uri) or {}
418         ) # empty dict still terminates the poll (present == done)
419
420     def _read_json(self, uri: str) -> dict:
421         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
422         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
423         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
424         try:
425             ref = parse_uri(uri)
426             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
427                 dst = os.path.join(td, ref.name or "obj.json")
428                 local = fetch(uri, dst, config=self._storage_config)
429                 with open(local, "r", encoding="utf-8") as f:

```

```

430         data: Any = json.load(f)
431         return data if isinstance(data, dict) else {}
432     except Exception:
433         LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
434         return {}
435

```

8. user (2026-07-07T15:48:27.846Z)

```

1     """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3     Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4     - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5     for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6     so it is abandoned, not joined ? it never blocks the process.
7     - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8     ``max_wait_sec`` deadline).
9
10    **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11    waiting (a fake clock advances instantly).
12
13    **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14    transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15    ``execution.policy`` logger to watch the poll trail.
16    """
17
18    from __future__ import annotations
19
20    import logging
21    import threading
22    import time as _time
23    from dataclasses import asdict, dataclass
24    from enum import Enum
25    from typing import Callable, Optional, TypeVar
26
27    T = TypeVar("T")
28
29    #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
30    LOG = logging.getLogger("execution.policy")
31
32
33    class WaitMode(str, Enum):
34        """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
35
36        WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
37        ABORT = "abort" # one attempt; on hang/fail, stop immediately
38        BLOCK = "block" # bounded in-process blocking wait (no reschedule)
39
40
41    class PolicyTimeout(TimeoutError):
42        """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
43
44
45    @dataclass(frozen=True)

```

```

46     class ExecutionPolicy:
47         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
48
49         mode: WaitMode = WaitMode.WAIT_AND_RESUME
50         poll_every_n_sec: float = 10.0
51         op_timeout_sec: float = (
52             120.0 # HARD per-attempt deadline ? the hang-killer that was missing
53         )
54         max_wait_sec: float = 1800.0 # total wall budget for a poll loop
55         max_retries: int = 5 # retries (=> max_retries+1 attempts) for failures/hangs
56         backoff_base_sec: float = 3.0 # exponential: backoff_base * 2**attempt
57         on_timeout: str = "reschedule" # reschedule | partial | fail (consumed by P2/P3)
58         resumable: bool = True
59
60         def to_dict(self) -> dict:
61             d = asdict(self)
62             d["mode"] = self.mode.value
63             return d
64
65         @classmethod
66         def from_dict(cls, d: dict) -> "ExecutionPolicy":
67             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
68             kw = {k: v for k, v in d.items() if k in known}
69             if "mode" in kw and not isinstance(kw["mode"], WaitMode):
70                 kw["mode"] = WaitMode(kw["mode"])
71             return cls(**kw)
72
73
74 #: The sane default (also what a job gets if it declares no policy).
75 DEFAULT_POLICY = ExecutionPolicy()
76
77
78 def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
79     """Deterministic exponential backoff (no jitter ? reproducible tests)."""
80     return policy.backoff_base_sec * (2**attempt)
81
82
83 def run_bounded(
84     fn: Callable[[], T],
85     policy: ExecutionPolicy = DEFAULT_POLICY,
86     *,
87     label: str = "op",
88     logger: Optional[logging.Logger] = None,
89     sleep: Callable[[float], None] = _time.sleep,
90 ) -> T:
91     """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
92
93     Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
94     last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
95     """
96     log = logger or LOG
97     attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
98     last_exc: Optional[BaseException] = None
99     for attempt in range(attempts):
100         box: dict = {}
101         done = threading.Event()
102
103         # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
104         # same iteration, but explicit binding keeps each attempt isolated and silences
B023.

```

```

105     def _runner(_box: dict = box, _done: threading.Event = done) -> None:
106         try:
107             _box["val"] = fn()
108         except (
109             BaseException
110             ) as e: # relay ANY error (incl. timeouts) to the caller thread
111             _box["exc"] = e
112         finally:
113             _done.set()
114
115     threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
116     if done.wait(timeout=policy.op_timeout_sec):
117         if "exc" in box:
118             last_exc = box["exc"]
119             log.info(
120                 "%s: attempt %d/%d failed: %s",
121                 label,
122                 attempt + 1,
123                 attempts,
124                 last_exc,
125             )
126         else:
127             if attempt:
128                 log.info(
129                     "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
130                 )
131             return box["val"] # type: ignore[return-value]
132     else:
133         last_exc = PolicyTimeout(
134             f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
135         )
136         log.warning(
137             "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
138             label,
139             attempt + 1,
140             attempts,
141             policy.op_timeout_sec,
142         )
143         if attempt < attempts - 1:
144             wait = _backoff_sec(policy, attempt)
145             log.debug("%s: backing off %.1fs before retry", label, wait)
146             sleep(wait)
147     assert last_exc is not None
148     log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
149     raise last_exc
150
151
152     def poll_until(
153         check: Callable[[], Optional[T]],
154         policy: ExecutionPolicy = DEFAULT_POLICY,
155         *,
156         label: str = "op",
157         logger: Optional[logging.Logger] = None,
158         sleep: Callable[[float], None] = _time.sleep,
159         monotonic: Callable[[], float] = _time.monotonic,
160         on_tick: Optional[Callable[[float], None]] = None,
161     ) -> T:
162         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
163         value, or raise
164         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
165         still in progress.
166         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
167         INFO/WARNING.
168         ``on_tick(elapsed_sec)`` (optional) fires once per still-waiting iteration ? the

```

```

caller's live
167     progress heartbeat (#482). Best-effort by contract: a raising callback is swallowed
168     (DEBUG-logged), never allowed to kill the wait it reports on.
169     """
170     log = logger or LOG
171     start = monotonic()
172     polls = 0
173     log.info(
174         "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
175         label,
176         policy.poll_every_n_sec,
177         policy.max_wait_sec,
178     )
179     while True:
180         result = check()
181         if result is not None:
182             log.info(
183                 "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
184             )
185             return result
186         elapsed = monotonic() - start
187         if elapsed >= policy.max_wait_sec:
188             log.warning(
189                 "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
190             )
191             raise PolicyTimeout(
192                 f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
193             )
194         polls += 1
195         log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
196         if on_tick is not None:
197             try:
198                 on_tick(elapsed)
199             except Exception: # noqa: BLE001 ? progress reporting must never kill the
wait
200                 log.debug("%s: on_tick callback failed", label, exc_info=True)
201                 sleep(policy.poll_every_n_sec)
202

```

9. assistant (2026-07-07T15:48:37.205Z)

Now let me look at the orchestration.py API caller and the worker `\_\_main\_\_.py` dispatch handler.

10. user (2026-07-07T15:48:38.544Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local:``/``
8     is an identity passthrough; ``s3:``/`` / ``gcs:``/`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    """

```

```

16
17     from __future__ import annotations
18
19     import argparse
20     import csv
21     import json
22     import logging
23     import os
24     import re
25     import sys
26     import tempfile
27     from typing import Any, List
28
29     from backend import storage
30     from backend.compute import ComputeJobSpec, get_compute_backend
31     from backend.config import (
32         StartupCheckError,
33         enforce_startup_checks,
34         load_config,
35         probe_cuda_available,
36     )
37     from backend.infrastructure.database import Database
38     from backend.pipeline.segmenters.base import segmenter_registry
39     from backend.pipeline.validators.base import registry as validator_registry
40     from backend.reporting import emit_indexer_report
41     from backend.results import Failure
42     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("backend.worker")
46
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
50         str."""
51         low = v.lower()
52         if low in ("true", "false"):
53             return low == "true"
54         for cast in (int, float):
55             try:
56                 return cast(v)
57             except ValueError:
58                 pass
59         return v
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
63         indexing.skip_ai_handoff=true)."""
64         for kv in sets or []:
65             key, sep, val = kv.partition("=")
66             if not sep:
67                 raise ValueError(f"--set expects k=v, got {kv!r}")
68             parts = key.split(".")
69             obj = config
70             for p in parts[:-1]:
71                 obj = getattr(obj, p)
72             setattr(obj, parts[-1], _coerce(val))
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
76         friendly
77         artifact (same schema as the rally-annotator labels)."""
78         with open(path, "w", newline="") as f:

```

```

78         w = csv.writer(f)
79         w.writerow(["start", "end", "shot_count", "ending_reason"])
80         for s in segments:
81             w.writerow(
82                 [
83                     f"{float(s.get('start_time', 0.0)):.3f}",
84                     f"{float(s.get('end_time', 0.0)):.3f}",
85                     s.get("shot_count", ""),
86                     s.get("ending_reason", s.get("shot_count_confidence", "")),
87                 ]
88             )
89
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {
106                             "start": float(s.get("start_time", 0.0)),
107                             "end": float(s.get("end_time", 0.0)),
108                         }
109                         for s in segments
110                     ],
111                 },
112                 f,
113                 indent=2,
114             )
115
116
117     def _serialize_and_push_outputs(
118         scratch: str,
119         out_uri: str,
120         video_id: str,
121         segments: List[dict],
122         status: str,
123         storage_cfg: dict,
124         video_uri: str = "",
125     ) -> List[str]:
126         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
127         push both to
128         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
129
130         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
131         timeout/abort leaves a
132         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
133         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
134         non-``success``
135         report as "no usable result" ? retry, and the restart-recovery poll waits for
136         report.json to
137         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
138         """
139         out_local = os.path.join(scratch, "outputs", video_id)
140         os.makedirs(out_local, exist_ok=True)
141         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
142         _write_report_json(
143             video_id,

```

```

139         segments,
140         status,
141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         ) # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181         * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
may still be
182         running (the shim best-effort cancels it).
183         * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
184         crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185         ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186         from backend.compute import RemoteExecutionFailed
187         from backend.infrastructure.execution_policy import PolicyTimeout
188
189         spec = ComputeJobSpec(
190             video_uri=args.video_uri,

```

```

191         out_uri=args.out_uri,
192         segmenter=args.segmenter,
193         config_overrides=tuple(args.set or ()),
194         skip_validation=bool(getattr(args, "skip_validation", False)),
195     )
196     try:
197         result = compute.run_infer(spec)
198     except RemoteExecutionFailed as e:
199         logger.error(
200             "[worker] remote execution terminated without a completion marker: %s", e
201         )
202         return 5
203     except PolicyTimeout as e:
204         logger.warning(
205             "[worker] remote compute did not complete (no marker within budget): %s", e
206         )
207         return 4
208     logger.info(
209         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210         result.video_id,
211         result.returncode,
212         result.outputs_uri,
213     )
214     return result.returncode
215
216
217 def _write_done_marker(
218     done_uri: str,
219     video_id: str,
220     returncode: int,
221     segment_count: int,
222     status: str,
223     storage_cfg: dict,
224     scratch: str,
225 ) -> None:
226     """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
227     for a remote
228     dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
229     return."""
230     try:
231         marker = {
232             "video_id": video_id,
233             "returncode": returncode,
234             "segment_count": segment_count,
235             "status": status,
236         }
237         local = os.path.join(scratch, "_done.json")
238         with open(local, "w", encoding="utf-8") as f:
239             json.dump(marker, f)
240         storage.put(local, done_uri, config=storage_cfg)
241         logger.info("[worker] wrote completion marker -> %s", done_uri)
242     except Exception as e: # never fail a good run because the marker push hiccuped
243         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
244
245
246 def cmd_infer(args: argparse.Namespace) -> int:
247     config = load_config(args.config) if args.config else load_config()
248     _apply_overrides(config, args.set)
249     if not _run_startup_checks(config):
250         return 2
251
252     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
253     process.
254     # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
255     through to the

```

```

252         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254         compute = get_compute_backend(config)
255         if compute is not None:
256             return _dispatch_remote(compute, args)
257
258         storage_cfg = config.storage.model_dump()
259
260         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261         os.makedirs(scratch, exist_ok=True)
262         config.output.output_dir = scratch
263
264         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265         local_video = storage.fetch(
266             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267         )
268         print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271         video_id = compute_video_id(local_video)
272
273         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282         if getattr(args, "skip_validation", False):
283             print(
284                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285                 "gate and already validated this input."
286             )
287             val_results, val_context = [], {}
288         else:
289             val_results, val_context = validator_registry.run_all_with_context(
290                 local_video, config
291             )
292         failures = [r for r in val_results if not r.passed]
293         db = Database(os.path.join(scratch, config.output.db_name))
294         db.add_video(
295             video_id,
296             local_video,
297             "failed" if failures else "processed",
298             [r.model_dump() for r in val_results],
299         )
300
301         def _fail(rc: int) -> int:
302             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the

```

```

303         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
305         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
308         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309         try:
310             _serialize_and_push_outputs(
311                 scratch,
312                 args.out_uri,
313                 video_id,
314                 [],
315                 "failed",
316                 storage_cfg,
317                 str(getattr(args, "video_uri", "") or ""),
318             )
319         except Exception as e: # never mask the real failure on an artifact-push hiccup
320             logger.warning("[worker] could not push failure artifacts: %s", e)
321         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322         if getattr(args, "done_uri", None):
323             _write_done_marker(
324                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
325             )
326         return rc
327
328     segments: List[dict] = []
329     segmenter_actual = None
330     segmentation_ran = False
331     status = "failed"
332     try:
333         if failures:
334             print(
335                 "[worker] validation FAILED: "
336                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
337             )
338             return _fail(2)
339         seg_name = args.segmenter or config.indexing.default_segmenter
340         segmenter_cls = segmenter_registry.get(seg_name)
341         if not segmenter_cls:
342             print(
343                 f"[worker] unknown segmenter: {seg_name!r} "
344                 f"(have {list(segmenter_registry.list_available())})"
345             )
346             return _fail(2)
347         segmenter_actual = seg_name
348         result = segmenter_cls(db, config).process_video(
349             video_id, local_video, max_frames=args.max_frames
350         )
351         segmentation_ran = True
352         if isinstance(result, Failure):
353             print(f"[worker] segmenter FAILED: {result.message}")
354             return _fail(3)
355         segments = result.value if hasattr(result, "value") else result
356         status = "success"
357         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359         # trajectories). Explain it so a run is analysable from the log alone (re-run -v

```

```

for more).
360         if not segments:
361             detector_impl = (
362                 os.environ.get("RALLY_DETECTOR_IMPL")
363                 or getattr(config.indexing, "detector_impl", None)
364                 or "auto"
365             )
366             logger.warning(
367                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
368                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
369                 "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
370                 "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
371                 "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
372                 "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
373                 "detections, or the input resolution differs from the tuned proxy
resolution. "
374                 "Re-run with -v for the native command + frame counts.",
375                 video_id,
376                 segmenter_actual,
377                 detector_impl,
378             )
379         finally:
380             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
381             emit_indexer_report(
382                 db=db,
383                 video_id=video_id,
384                 video_path=local_video,
385                 config=config,
386                 val_results=val_results,
387                 val_context=val_context,
388                 segmenter_requested=args.segmenter,
389                 segmenter_actual=segmenter_actual,
390                 was_reingest=False,
391                 segmentation_ran=segmentation_ran,
392             )
393
394         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
395         pushed = _serialize_and_push_outputs(
396             scratch,
397             args.out_uri,
398             video_id,
399             segments,
400             status,
401             storage_cfg,
402             str(getattr(args, "video_uri", "") or ""),
403         )
404
405         print(
406             f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
407         )
408         for u in pushed:
409             print("    ", u)
410
411         # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
412         # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
413         # write their own marker via _fail() above, so the dispatcher fails FAST either way

```

```

(no 30-min hang
414     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
415     if getattr(args, "done_uri", None):
416         _write_done_marker(
417             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418         )
419     return 0
420
421
422     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
423     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
424     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
425
426     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
427     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
428     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
429     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
430
431
432     def _label_clip_name(fname: str) -> str:
433         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
434
435         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
436         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
437
438         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
stamped)
439         * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
stem)
440         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
441         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
442
443         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
444         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
445         """
446         name = fname.replace(".rallies.csv", "").replace(".csv", "")
447         return _LABEL_DATE_PREFIX.sub("", name)
448
449
450     def _probe_video_fps_width(path: str):
451         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
452
453         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
454         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
455         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
456         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
457         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
458         rather than guesses.
459         """

```

```

460 import json
461 import subprocess
462
463 try:
464     out = subprocess.check_output(
465         [
466             ffprobe_bin(),
467             "-v",
468             "error",
469             "-select_streams",
470             "v:0",
471             "-show_entries",
472             "stream=r_frame_rate,width",
473             "-of",
474             "json",
475             path,
476         ],
477         stderr=subprocess.DEVNULL,
478     ).decode()
479     st = (json.loads(out).get("streams") or [{}])[0]
480     num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
481     fps = float(num) / float(den or "1")
482     width = float(st.get("width") or 0)
483     if fps > 0 and width > 0:
484         return fps, width
485 except (
486     subprocess.SubprocessError,
487     ValueError,
488     KeyError,
489     OSError,
490     json.JSONDecodeError,
491 ):
492     pass
493 return None
494
495
496 def _resolve_train_detector(args: argparse.Namespace, config: Any):
497     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
498     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
499     stub=CPU mock). Returns the runner, or prints an error + returns None.
500
501     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
502     the
503     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
504     has
505     no shuttle detector and is rejected.
506     """
507     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
508
509     seg_cls = segmenter_registry.get(args.segmenter)
510     if not seg_cls:
511         print(
512             f"[worker] unknown segmenter: {args.segmenter!r} "
513             f"(have {list(segmenter_registry.list_available())})"
514         )
515         return None
516     seg = seg_cls(None, config)
517     if not isinstance(seg, TrajectoryHybridSegmenter):
518         print(
519             f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
520             f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
521         )
522         return None
523     try:

```

```

522         runner, _ = seg._resolve_runner(config.indexing)
523     except NotImplementedError as e:
524         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
525         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
526         traceback.print(f"[worker] detector not available: {e}")
527         return None
528     ok, msg = runner.healthcheck()
529     if not ok:
530         print(f"[worker] detector environment not ready: {msg}")
531         return None
532     print(f"[worker] detector ready ({args.segmenter}): {msg}")
533     return runner
534
535
536 def cmd_train(args: argparse.Namespace) -> int:
537     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
538     shell out to training.gen0.harness (backend must NOT import training) -> push the artifact
539     bundle.
540     Two trajectory sources (the train pipeline + harness are identical either way):
541     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
542     synthetic shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
543     CI.
544     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
545     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
546     This is the M3.5 "first real training" path; only the trajectory source flips.
547     """
548     import subprocess
549
550     config = load_config(args.config) if args.config else load_config()
551     _apply_overrides(config, args.set)
552     if not _run_startup_checks(config):
553         return 2
554     storage_cfg = config.storage.model_dump()
555
556     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
557     labels_dir = os.path.join(scratch, "labels")
558     traj_dir = os.path.join(scratch, "trajectories")
559     videos_dir = os.path.join(scratch, "videos")
560     os.makedirs(labels_dir, exist_ok=True)
561     os.makedirs(traj_dir, exist_ok=True)
562
563     corpus = args.corpus_uri.rstrip("/")
564     label_uris = sorted(
565         u
566         for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
567         if u.lower().endswith(".csv")
568     )
569     if len(label_uris) < 2:
570         print(
571             f"[worker] need >=2 labeled videos for leave-one-video-out; found
572             {len(label_uris)} "
573             f"under {corpus}/labels/"
574         )
575         return 2
576
577     # Real-detector source: resolve the runner once + index the bucket's videos/ by
578     stem.
579     runner = None
580     video_by_stem: dict = {}
581     if args.trajectory_source == "detector":

```

```

579         os.makedirs(videos_dir, exist_ok=True)
580         runner = _resolve_train_detector(args, config)
581         if runner is None:
582             return 2
583         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
584             vname = storage.parse_uri(vuri).name
585             if not vname.lower().endswith(_VIDEO_EXTS):
586                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
587             video_by_stem[os.path.splitext(vname)[0]] = vuri
588
589         print(f"[worker] trajectory source: {args.trajectory_source}")
590         specs: List[dict] = []
591         for uri in label_uris:
592             fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
593             name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
594             local_label = storage.fetch(
595                 uri, os.path.join(labels_dir, fname), config=storage_cfg
596             )
597             if args.trajectory_source == "detector":
598                 vuri = video_by_stem.get(name)
599                 if not vuri:
600                     print(
601                         f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
602                     )
603                     continue
604                 local_video = storage.fetch(
605                     vuri,
606                     os.path.join(videos_dir, storage.parse_uri(vuri).name),
607                     config=storage_cfg,
608                 )
609                 video_id = compute_video_id(local_video)
610                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
611                 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
612                 meta = _probe_video_fps_width(local_video)
613                 if meta is None:
614                     print(
615                         f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
616                     )
617                     continue
618                 fps, frame_width = meta
619                 traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
620                 if not traj:
621                     print(
622                         f"[worker] detector produced no trajectory for {name!r} ? skipping."
623                     )
624                     continue
625                 specs.append(
626                     {
627                         "name": name,
628                         "trajectory": traj,
629                         "labels": local_label,
630                         "fps": fps,
631                         "frame_width": frame_width,
632                     }
633                 )
634             else:
635                 from backend.pipeline.detectors.stub_runner import (
636                     synth_trajectory_from_labels,
637                 )
638
639                 traj = os.path.join(traj_dir, f"{name}_traj.csv")

```

```

640         synth_trajectory_from_labels(
641             local_label, traj, fps=args.fps, frame_width=args.frame_width
642         )
643         specs.append(
644             {
645                 "name": name,
646                 "trajectory": traj,
647                 "labels": local_label,
648                 "fps": args.fps,
649                 "frame_width": args.frame_width,
650             }
651         )
652
653     if len(specs) < 2:
654         print(
655             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
656         )
657         return 2
658     manifest_path = os.path.join(scratch, "manifest.json")
659     with open(manifest_path, "w", encoding="utf-8") as f:
660         json.dump(specs, f, indent=2)
661     src_label = (
662         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
663     )
664     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
665
666     out_dir = os.path.join(scratch, "artifacts")
667     cmd = [
668         sys.executable,
669         "-m",
670         "training.gen0.harness",
671         "--manifest",
672         manifest_path,
673         "--out",
674         out_dir,
675         "--version",
676         args.version,
677     ]
678     if args.floor:
679         floor_local = (
680             storage.fetch(
681                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
682             )
683             if "://" in args.floor
684             else args.floor
685         )
686         cmd += ["--floor", floor_local]
687     print("[worker] training:", " ".join(cmd))
688     rc = subprocess.run(cmd).returncode
689     if rc != 0:
690         print(f"[worker] gen0 harness failed (rc={rc})")
691         return rc
692
693     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
694     bundle = os.path.join(out_dir, args.version)
695     out_prefix = args.out_uri.rstrip("/")
696     pushed = []
697     for fn in sorted(os.listdir(bundle)):
698         uri = f"{out_prefix}/weights/{args.version}/{fn}"
699         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
700         pushed.append(uri)
701     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
702     for u in pushed:
703         print("    ", u)

```

```

704         return 0
705
706
707     def build_parser() -> argparse.ArgumentParser:
708         p = argparse.ArgumentParser(
709             prog="python -m backend.worker",
710             description="Storage-aware worker: pull ? run pipeline ? push.",
711         )
712         p.add_argument(
713             "-v",
714             "--verbose",
715             action="store_true",
716             help="DEBUG logging (the native detector command, frame counts, per-stage
717 detail)",
718         )
719         sub = p.add_subparsers(dest="cmd", required=True)
720
721         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
722         pi.add_argument(
723             "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
724         )
725         pi.add_argument(
726             "--out-uri",
727             required=True,
728             help="output prefix (outputs/<video_id>/? is appended)",
729         )
730         pi.add_argument(
731             "--segmenter", default=None, help="default: indexing.default_segmenter"
732         )
733         pi.add_argument(
734             "--config", default=None, help="config.json path (default: ./config.json)"
735         )
736         pi.add_argument(
737             "--scratch", default=None, help="local scratch dir (default: a temp dir)"
738         )
739         pi.add_argument("--max-frames", type=int, default=None)
740         pi.add_argument(
741             "--done-uri",
742             default=None,
743             help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
744             "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
745             "Default-OFF: omit it for the normal local run.",
746         )
747         pi.add_argument(
748             "--set",
749             action="append",
750             default=[],
751             metavar="k=v",
752             help="config override, e.g. --set indexing.skip_ai_handoff=true",
753         )
754         pi.add_argument(
755             "--skip-validation",
756             action="store_true",
757             help="skip the worker's re-validation: the dispatcher (control-plane) already
758 ran the "
759 "validators with its resolved config and only dispatches on PASS, so re-
760 validating here is "
761 "redundant, decodes the video again, and can contradict the gate. Default-OFF:
762 the direct "
763 "`worker infer` CLI still validates.",
764         )
765         pi.set_defaults(func=cmd_infer)
766
767         pt = sub.add_parser(
768             "train",

```

```

765         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
766     )
767     pt.add_argument(
768         "--corpus-uri",
769         required=True,
770         help="prefix containing labels/ (+ videos/ for "
771         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
772     )
773     pt.add_argument(
774         "--out-uri",
775         required=True,
776         help="output prefix (weights/<version>/ is appended)",
777     )
778     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
779     pt.add_argument(
780         "--trajectory-source",
781         choices=["synth", "detector"],
782         default="synth",
783         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
784         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
785         "(needs videos/ in the corpus; honours detector_impl).",
786     )
787     pt.add_argument(
788         "--segmenter",
789         default="wasb_hybrid",
790         help="trajectory hybrid whose detector is used for --trajectory-source
791         detector",
792     )
793     pt.add_argument(
794         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
795     )
796     pt.add_argument("--config", default=None)
797     pt.add_argument("--scratch", default=None)
798     pt.add_argument(
799         "--fps",
800         type=float,
801         default=30.0,
802         help="synth trajectory fps (detector probes the video)",
803     )
804     pt.add_argument(
805         "--frame-width",
806         type=float,
807         default=1280.0,
808         help="synth trajectory frame width (detector probes the video)",
809     )
810     pt.add_argument("--set", action="append", default=[], metavar="k=v")
811     pt.set_defaults(func=cmd_train)
812     return p
813
814     def _setup_logging(verbose: bool) -> None:
815         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
816         diagnostics are
817         VISIBLE ? the worker previously ran with no logging config, so every
818         segmenter/detector log
819         (detector choice, 'Running native WASB inference', availability failures) was
820         silently dropped.
821         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
822         (the CLI
823         entry), not at import, so importing the module never reconfigures a caller's
824         logging."""
825         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
826         library
827         # configured logging at import), which would leave the level at WARNING and re-
828         suppress the

```

```

822         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
823         logging.basicConfig(
824             level=logging.DEBUG if verbose else logging.INFO,
825             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
826             datefmt="%H:%M:%S",
827             force=True,
828         )
829
830
831     def main(argv=None) -> int:
832         args = build_parser().parse_args(argv)
833         _setup_logging(getattr(args, "verbose", False))
834         return args.func(args)
835
836
837     if __name__ == "__main__":
838         sys.exit(main())
839

```

11. assistant (2026-07-07T15:48:41.566Z)

Now let me examine the orchestration.py caller and base.py.

12. user (2026-07-07T15:48:42.562Z)

```

1         """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3         COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4         (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5         core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6         (cloud-serving) ? without the core ever branching on the profile (D1).
7
8         The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9         non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10        existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11        """
12
13        from __future__ import annotations
14
15        from abc import ABC, abstractmethod
16        from dataclasses import dataclass, field
17        from typing import Callable, Optional
18
19
20        class RemoteExecutionFailed(RuntimeError):
21            """A dispatched remote job's execution reached a TERMINAL state without ever writing
its
22            completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image
argparse-rejecting
23            an unknown flag) before the marker push, or the marker upload itself failed on an
otherwise
24            Succeeded run.
25
26            Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL
be
27            running): this says the execution is DEFINITELY over and no marker will ever
appear, so the
28            dispatcher must stop bucket-polling immediately instead of burning the full
29            ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message
carries the
30            job + execution name + terminal state + the ``gcloud`` command to read the execution

```

```

log, so
31         the whole "worker crashed before the done-marker" class fails fast AND
diagnostically."""
32
33
34     @dataclass(frozen=True)
35     class ComputeJobSpec:
36         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
37         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
38         deterministic regardless of where the job ran."""
39
40         video_uri: str # input (gs://? for cloud; a path/local:// for local)
41         out_uri: str # output ROOT (gs://? bucket for cloud)
42         segmenter: Optional[str] = None
43         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
44         config_overrides: tuple[str, ...] = ()
45         # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving
control-plane) is
46         # the authoritative validation gate: it runs the SAME validators with ITS resolved
config and only
47         # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video
decode, and
48         # (c) outright WRONG when the worker's config resolves different thresholds ? it can
REJECT a clip
49         # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no
control-plane)
50         # validating exactly as before.
51         skip_validation: bool = False
52
53
54     @dataclass(frozen=True)
55     class ComputeResult:
56         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
57         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
58         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
59         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
60
61         returncode: int
62         outputs_uri: str = ""
63         video_id: str = ""
64         report: dict = field(default_factory=dict)
65         # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
66         # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
67         # executions describe <execution>). Empty for the in-process path / backends without
one.
68         execution: str = ""
69
70
71     class ComputeBackend(ABC):
72         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
73         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
74         execution identically."""
75
76         @abstractmethod

```

```

77         def run_infer(
78             self,
79             spec: ComputeJobSpec,
80             *,
81             on_wait: "Callable[[float], None] | None" = None,
82         ) -> ComputeResult:
83             """Run ``spec`` to completion and return its ``ComputeResult``.
84
85             ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
86             still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
87             so a
88             long GPU run doesn't look frozen. Implementations without a wait loop may ignore
89             it."""

```

13. user (2026-07-07T15:48:44.418Z)

```

4:Cloud Run dispatch + ingest path (`_maybe_dispatch_remote` / `_ingest_remote_segments`),
98:     except Exception:
201:     except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
281:     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch path
357:     except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
370:     except Exception: # noqa: BLE001 ? recovery nicety, never a failure
391: def _maybe_dispatch_remote(
511:     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure,
    not a 500
589:     result = compute.run_infer(
593:     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
    not a 500
616:     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not 500
    or leak rows
703:     except Exception:
710:     except Exception:
728:     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
    fail).
805:     remote = _maybe_dispatch_remote(
823:     except Exception:
918:     except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must not
    block
1065:     except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
    FAILURE, not a crash
1091:     except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch already
    succeeded

```

14. assistant (2026-07-07T15:48:46.970Z)

Let me examine the orchestration.py `\_maybe\_dispatch\_remote` and the `run\_infer` call site around lines 391-620.

15. user (2026-07-07T15:48:48.340Z)

```

560     wasb_cfg = getattr(idx_cfg, "wasb", None)
561     if wasb_cfg is not None:
562         decode_backend = getattr(wasb_cfg, "decode_backend", None)
563         if decode_backend:
564             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
565         batch_size = getattr(wasb_cfg, "batch_size", None)
566         if batch_size:
567             overrides.append(f"indexing.wasb.batch_size={batch_size}")
568         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
569         if prefetch:
570             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
571     # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
    authoritative

```

```

572         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
573         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
574         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
575         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
576         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
577         spec = ComputeJobSpec(
578             video_uri=req.video_path,
579             out_uri=out_root,
580             segmenter=seg_name,
581             config_overrides=tuple(overrides),
582             skip_validation=True,
583         )
584         try:
585             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
586             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
587             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
588             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
589             result = compute.run_infer(
590                 spec,
591                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
592             )
593             except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
594                 logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
595                 return _failed(f"cloud dispatch error: {e}", True)
596             if result.returncode != 0:
597                 return _failed(f"cloud job failed (returncode {result.returncode}).", True)
598             if result.video_id and result.video_id != video_id:
599                 # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
600                 # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
601                 logger.warning(
602                     "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
603                     "and the worker hash; ingesting under the local id.",
604                     result.video_id,
605                     video_id,
606                 )
607
608             notify("ingesting (cloud_run)")
609             storage_dict = (
610                 storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
611                 if hasattr(storage_cfg, "model_dump")
612                 else {}
613             )
614             try:
615                 _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
616             except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
617                 logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
618                 return _failed(
619                     f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
620                     True,
621                 )
622
623             # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.

```

```

624     segments = [
625         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
626     ]
627     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
628     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
629     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
630     provenance = {
631         "path": "cloud_run",
632         "requested": req.compute,
633         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
634         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
635         "execution": result.execution,
636         "outputs_uri": result.outputs_uri,
637     }
638     logger.info(
639         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
640         result.execution,
641         provenance["job"],
642         provenance["region"],
643         result.outputs_uri,
644         video_id,
645     )
646     return (
647         {
648             "video_id": video_id,
649             "status": "success",
650             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
651             "results": [r.model_dump() for r in val_results],
652             "play_segments": segments,
653             "compute": provenance,
654         },
655         True,
656     )
657
658
659     def _execute_processing(
660         config=None, db=None, req=None, progress=None
661     ) -> Dict[str, Any]:
662         """The full hash ? validate ? segment ? report pipeline for one video.
663
664         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
665         module-level ``global_config``/``db_instance`` (backward-compatible with existing
666         tests that pass only ``req``). Route handlers always pass explicit params; older
667         test code that calls ``_execute_processing(req)`` still works.
668
669         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
670         Raises on unexpected internal errors ? the caller records those as a job failure
671         """
672         import backend.main as _main # call-time seam: main's globals/model stay patchable
673
674         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
675         # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
676         if config is not None and not isinstance(config, AppConfig):
677             # Old signature: _execute_processing(req, progress=None)
678             if req is None and db is not None and isinstance(db, _main.ProcessRequest):
679                 req = db
680                 db = None
681             elif req is None and isinstance(config, _main.ProcessRequest):
682                 req = config

```

```

683         config = None
684     # Resolve config/db from module globals when not passed explicitly
685     if config is None:
686         config = _main.global_config
687     if db is None:
688         db = _main.db_instance
689

```

16. assistant (2026-07-07T15:50:01.229Z)

Let me check the storage layer's `exists` implementation to assess consistency-related race risk in the fast-fail path.

17. user (2026-07-07T15:50:03.612Z)

```

backend\storage\gdrive.py:135:     def exists(self, ref) -> bool:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-136-         return self._local.exists(self._local_ref(ref))
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-137-
backend\storage\gdrive.py-138:     def stat(self, ref) -> StorageStat:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-139-         self._require_present(ref)
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-140-         return self._local.stat(self._local_ref(ref))
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-141-
backend\storage\gdrive.py-142:     def list(self, prefix) -> Iterator[StorageRef]:
backend\storage\gdrive.py-143-         for r in self._local.list(self._local_ref(prefix)):
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-144-             uri = self._to_gdrive_uri(r.key)
backend\storage\gdrive.py-145-             yield StorageRef("gdrive", "", uri[len("gdrive://")
:], uri)
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-146-
backend\storage\gdrive.py-147:     def delete(self, ref) -> None:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive.py-148-         self._local.delete(self._local_ref(ref))
--
backend\storage\base.py:44:     def exists(self, ref: StorageRef) -> bool: ...
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-45-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-46-
@abstractmethod
backend\storage\base.py-47:     def stat(self, ref: StorageRef) -> StorageStat: ...
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-48-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-49-
@abstractmethod
backend\storage\base.py-50:     def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-51-
"""Yield a ``StorageRef`` per object under ``prefix``."""
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-52-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-53-
@abstractmethod
backend\storage\base.py-54:     def delete(self, ref: StorageRef) -> None: ...
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-55-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-56-
def signed_url(
backend\storage\base.py-57-     self, ref: StorageRef, *, ttl: "int | None" = None, method:
str = "GET"
backend\storage\base.py-58- ) -> str:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-59-
"""A time-limited URL handing the bytes to a browser/LLM. ``ttl=None`` (the default)
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-60-
means "use the backend's configured ``signed_url_ttl``" ? so operator config wins
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\storage\base.py-61-

```

```

unless a caller explicitly needs a different expiry (#469). Not every backend can
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-62-
sign (`local` cannot; Drive only emulates) ? callers must treat
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-63-
`NotImplementedError` as expected and fall back to `fetch_to_local`. """
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\base.py-64-
raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
--
backend\storage\gcs.py:72:     def exists(self, ref) -> bool:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-73-
return bool(self._blob(ref).exists())
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-74-
backend\storage\gcs.py-75:     def stat(self, ref) -> StorageStat:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-76-
b = self._blob(ref)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-77-
b.reload()
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-78-
return StorageStat(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-79-
size=int(b.size or 0),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-80-
modified=b.updated,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-81-
etag=b.etag,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-82-
content_type=b.content_type,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-83-
md5=_md5_hex(getattr(b, "md5_hash", None)),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-84-
)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-85-
backend\storage\gcs.py-86:     def list(self, prefix) -> Iterator[StorageRef]:
backend\storage\gcs.py-87:         for blob in self._client.list_blobs(prefix.bucket,
prefix=prefix.key):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-88-
yield StorageRef(
backend\storage\gcs.py-89:             "gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-90-
)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\gcs.py-91-
backend\storage\gcs.py-92:     def delete(self, ref) -> None:
--
backend\storage\gdrive_api.py:209:     def exists(self, ref) -> bool:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-210-         return self._resolve_id(ref.key) is not None
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-211-
backend\storage\gdrive_api.py-212:     def stat(self, ref) -> StorageStat:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-213-         file_id = self._resolve_id(ref.key)
backend\storage\gdrive_api.py-214-         if not file_id:
backend\storage\gdrive_api.py-215-             raise FileNotFoundError(f"gdrive-api: {ref.uri!r}
not found")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-216-         meta = (
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-217-             self._service.files()
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-218-             .get(fileId=file_id, fields="size,
modifiedTime, md5Checksum, mimeType")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-219-             .execute()
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-

```

```

indexer\backend\storage\gdrive_api.py-220-        )
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-221-        modified = None
backend\storage\gdrive_api.py-222-        if meta.get("modifiedTime"):
backend\storage\gdrive_api.py-223-            # Drive returns RFC-3339 (e.g.
"2026-06-22T10:00:00.000Z"); normalize the trailing Z.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-224-        modified = datetime.fromisoformat(
backend\storage\gdrive_api.py-225-            meta["modifiedTime"].replace("Z", "+00:00")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-226-        )
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-227-        return StorageStat(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-228-            size=int(meta.get("size") or 0),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\gdrive_api.py-229-            modified=modified,
--
backend\storage\s3.py:62:    def exists(self, ref) -> bool:
backend\storage\s3.py-63-        try:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-64-        self._client.head_object(Bucket=ref.bucket, Key=ref.key)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-65-        return True
backend\storage\s3.py-66-        except Exception: # noqa: BLE001 - any boto/client error means
"not found / unreachable"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-67-        return False
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-68-
backend\storage\s3.py-69-    def stat(self, ref) -> StorageStat:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-70-    h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-71-    return StorageStat(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-72-        size=int(h.get("ContentLength", 0)),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-73-        modified=h.get("LastModified"),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-74-        etag=h.get("ETag"),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-75-        content_type=h.get("ContentType"),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-76-    )
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-77-
backend\storage\s3.py-78-    def list(self, prefix) -> Iterator[StorageRef]:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-79-    paginator = self._client.get_paginator("list_objects_v2")
backend\storage\s3.py-80-    for page in paginator.paginate(Bucket=prefix.bucket,
Prefix=prefix.key):
backend\storage\s3.py-81-        for obj in page.get("Contents", []) or []:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\s3.py-82-        key = obj["Key"]
--
backend\storage\__init__.py:200: def exists(uri: str, *, config: Optional[Dict[str, Any]] = None)
-> bool:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-201-    """True if the object at ``uri`` exists (a bucket
object or a local file)."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-202-    ref = parse_uri(uri)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-203-    return bool(get_storage(ref.backend,
config=config).exists(ref))
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-

```

```

indexer\backend\storage\__init__.py-204-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-205-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-206-def signed_url(
backend\storage\__init__.py-207-     uri: str,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-208-     *,
backend\storage\__init__.py-209-     config: Optional[Dict[str, Any]] = None,
backend\storage\__init__.py-210-     ttl: Optional[int] = None,
backend\storage\__init__.py-211-     method: str = "GET",
backend\storage\__init__.py-212-) -> str:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-213-     """A time-limited signed URL for ``uri`` ? the
client fetches bytes DIRECTLY from the store, off
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-214-     the app's response path (so a large object isn't
streamed back through the server; #463). Backends
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-215-     without object signing (e.g. local) raise
`NotImplementedError`.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-216-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-217-     ``ttl=None`` (the default) defers to the backend's
configured ``signed_url_ttl`` from
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-218-     ``config`` ? a hardcoded default here would silently
shadow the operator's setting (#469).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-219-     An explicit ``ttl`` overrides it for callers that
need a specific expiry."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\storage\__init__.py-220-     ref = parse_uri(uri)
--
backend\storage\local.py:50:     def exists(self, ref) -> bool:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-51-
return os.path.exists(self._path(ref))
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-52-
backend\storage\local.py-53-     def stat(self, ref) -> StorageStat:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-54-
st = os.stat(self._path(ref))
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-55-
return StorageStat(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-56-
size=st.st_size, modified=datetime.fromtimestamp(st.st_mtime)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-57-
)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-58-
backend\storage\local.py-59-     def list(self, prefix) -> Iterator[StorageRef]:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-60-
base = self._path(prefix)
backend\storage\local.py-61-         if os.path.isdir(base):
backend\storage\local.py-62-             for root, _dirs, files in os.walk(base):
backend\storage\local.py-63-                 for fn in sorted(files):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-64-
full = os.path.join(root, fn)
backend\storage\local.py-65-                 yield StorageRef("local", "", full,
f"local://{full}")
backend\storage\local.py-66-         elif os.path.exists(base):
backend\storage\local.py-67-             yield StorageRef("local", "", base, f"local://{base}")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-68-
backend\storage\local.py-69-     def delete(self, ref) -> None:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\local.py-70-
path = self._path(ref)

```

18. assistant (2026-07-07T15:50:43.813Z)

GCS `exists()` uses `blob.exists()` (a strongly-consistent object GET), so the race window is effectively closed; the double-read further mitigates. Let me run the targeted test module to empirically confirm the fast-fail behavior and check for any wiring issue.

19. user (2026-07-07T15:50:54.556Z)

```
..... [100%]  
31 passed in 6.32s
```

20. assistant (2026-07-07T15:50:57.754Z)

Let me confirm there are exactly the callers I expect and check the `\_\_init\_\_.py` factory wiring.

21. user (2026-07-07T15:50:59.378Z)

```
backend\orchestration.py:589:         result = compute.run_infer(  
backend\worker\_main_.py:197:         result = compute.run_infer(spec)
```

22. user (2026-07-07T15:50:59.752Z)

```
1      """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.  
2  
3      COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a  
4      ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and  
5      ``None``  
6      otherwise ? ``None`` means "run the worker's existing in-process path" (today's  
behaviour, byte  
7      identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /  
8      ``cpu_mock`` nothing  
9      changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when  
needed).  
10     """  
11  
12     from __future__ import annotations  
13  
14     import dataclasses  
15     import os  
16     from typing import Any, Optional  
17  
18     from backend.compute.base import (  
19         ComputeBackend,  
20         ComputeJobSpec,  
21         ComputeResult,  
22         RemoteExecutionFailed,  
23     )  
24     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy  
25  
26     __all__ = [  
27         "ComputeBackend",  
28         "ComputeJobSpec",  
29         "ComputeResult",  
30         "RemoteExecutionFailed",  
31         "get_compute_backend",  
32     ]  
33  
34     def _storage_dict(config: Any) -> dict:  
35         storage = getattr(config, "storage", None)  
36         if storage is None:  
37             return {}  
38         if hasattr(storage, "model_dump"):
```

```

38         return storage.model_dump() # type: ignore[no-any-return]
39     return dict(storage) if isinstance(storage, dict) else {}
40
41
42     def get_compute_backend(
43         config: Any,
44         *,
45         run_client: Optional[Any] = None,
46         policy: ExecutionPolicy = DEFAULT_POLICY,
47         token: Optional[str] = None,
48         target_override: Optional[str] = None,
49     ) -> Optional[ComputeBackend]:
50         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
51         in-process (the default-OFF path).
52
53         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
54         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
55         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
56         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
57
58         ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
59         per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
60         server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
61         """
62         target = (
63             target_override
64             if target_override is not None
65             else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
66         )
67         if target != "cloud_run":
68             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
69
70         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
71         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
72
73         # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
74         # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
75         # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the
old
76         # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
77         # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
78         if policy is DEFAULT_POLICY:
79             max_wait = getattr(
80                 getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
81             )
82             if max_wait:
83                 policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
84
85         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
86         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
87         project = os.environ.get("GOOGLE_CLOUD_PROJECT")
88         client = run_client if run_client is not None else GoogleCloudRunJobClient()
89         return CloudRunCompute(
90             run_client=client,

```

```

91         job_name=job_name,
92         region=region,
93         project=project,
94         storage_config=_storage_dict(config),
95         policy=policy,
96         token=token,
97     )
98

```

23. assistant (2026-07-07T15:51:52.469Z)

I've completed a thorough review of the fast-fail control flow. Let me verify my final reasoning on the two exception-propagation chains and confirm no intermediate catch swallows `RemoteExecutionFailed`.

The analysis is complete. Here is my assessment of the five lens points:

(1) Marker checked first, always wins (incl. empty dict): `\_poll\_check` reads the marker first; `\_read\_marker` returns `{}` (not `None`) for a present-but-empty/unreadable object (`self.\_read\_json(done\_uri) or {}`), and `{}` is not `None` ? returned immediately, before any status probe. An empty marker resolves the poll and is downstream treated as rc-1 failure in `run\_infer`. Status is never consulted when a marker exists. Correct.

(2) Post-status race re-check: After a terminal `state`, the code re-reads the marker exactly once and returns it if present, else raises. Mirrors `\_handle\_poll\_timeout`'s rescue. `test\_terminal\_status\_with\_marker\_written\_in\_race\_resolves\_via\_marker` confirms `exists` is called exactly twice. Correct.

***(3) Propagation past `except PolicyTimeout`:** `\_poll\_until` calls `result = check()` completely unguarded (only `on\_tick` is wrapped), so `RemoteExecutionFailed` propagates out. It is a `RuntimeError`; `PolicyTimeout` is a `TimeoutError` ? disjoint hierarchies ? so `except PolicyTimeout` in `run\_infer` cannot catch it, and it escapes `run\_infer`. Confirmed.

(4) rc 5 vs rc 4 coherence + orchestration path: Both callers of `run\_infer` handle it. `worker/\_\_\_main\_\_\_.dispatch\_remote` catches `RemoteExecutionFailed` ? rc 5 (distinct from the `PolicyTimeout` ? rc 4), and the two `except` clauses are order-independent because the types are disjoint. `orchestration.\_maybe\_dispatch\_remote` catches it via the broad `except Exception` ? `\_failed("cloud dispatch error: {e}", True)`, surfacing the full diagnostic (job, execution, terminal state, `gcloud` command) fast instead of after ~65 min. Coherent.

(5) Best-effort swallow cannot fail a healthy run nor mask a terminal failure: `\_terminal\_execution\_state` swallows any probe exception ? `None` ? keep polling; a flaky/raising status API degrades to today's marker-only path (`test\_status\_probe\_error\_is\_swallowed...` ? `PolicyTimeout`, not `RemoteExecutionFailed`). The real client's probe gates on `completion\_time.seconds`, so it can only report terminal once the container has actually exited; because the worker's `storage.put(marker)` blocks until upload completes *before* the process exits, and GCS `exists()` is a strongly-consistent object GET (`blob.exists()`), a healthy Succeeded run whose marker was written is seen by the first or the race re-check ? no spurious fast-fail. The only Succeeded-without-marker case is a genuinely failed marker upload, correctly surfaced as failure. A down status API loses the optimization but still yields rc-4, never a false success.

No defect-grade issues found.

24. user (2026-07-07T15:52:05.820Z)

Structured output provided successfully